Ynov Campus Aix - B2 Informatique - Conteneurisation Docker

Projet fil rouge
Partie 2 : le site passe en production

Rapport de TP

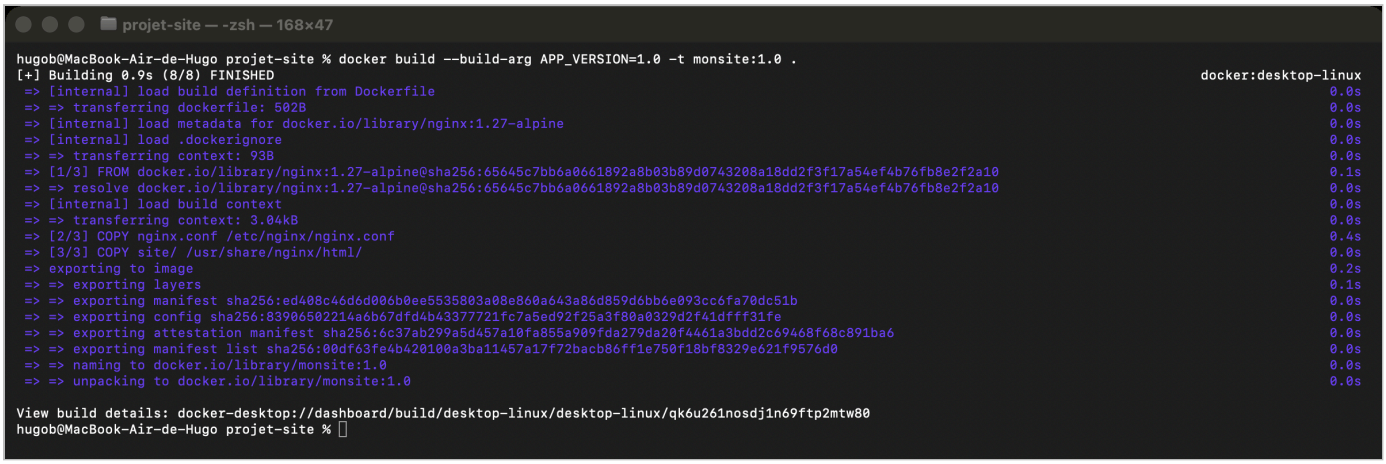
Hugo BERTON

Rendu le 9 octobre 2026

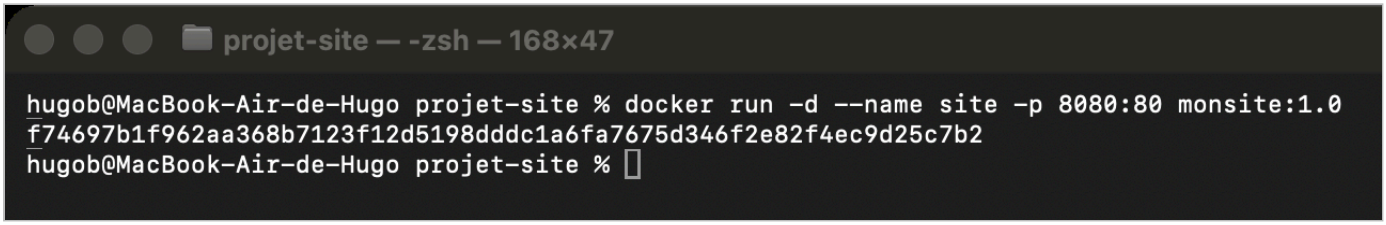
Intervenant : J. Novaro

Étape 1 - L'image du site

Je suis reparti de mon dossier `projet-site/` de la partie 1. J'ai rangé la page dans `site/index.html` et gardé le `nginx.conf` récupéré avec `docker cp`. J'ai ensuite écrit un `Dockerfile` basé sur `nginx:1.27-alpine` (contenu complet en annexe) et un `.dockerignore`.



Construction de l'image monsite:1.0 avec APP_VERSION=1.0 : on voit les deux COPY (nginx.conf et site/).



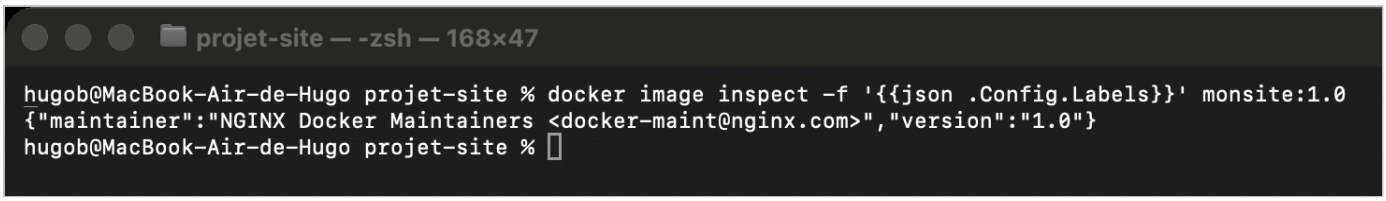
Lancement du conteneur site publié sur le port 8080.



Ma page personnalisée (Boulangerie du Cours) affichée dans le navigateur sur http://localhost:8080.



docker ps : le conteneur site est « Up (healthy) », le HEALTHCHECK (wget toutes les 10 s) passe.



docker image inspect : le label version vaut bien 1.0 (valeur passée avec --build-arg APP_VERSION=1.0).

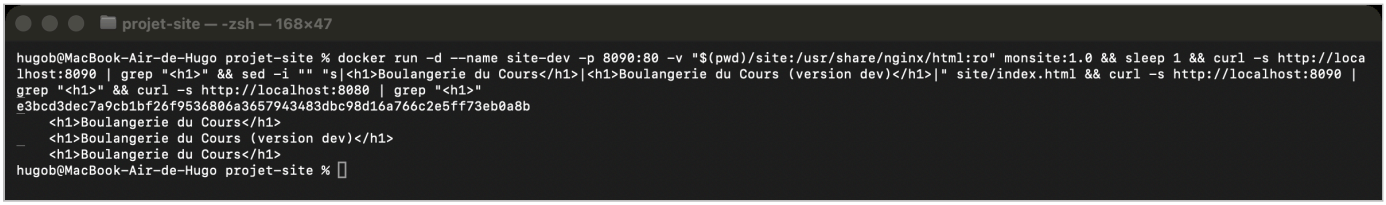


Suppression du conteneur site puis relance avec la même commande : la page de la boulangerie est toujours servie.

> **Question : pourquoi n'est-il pas nécessaire de définir CMD ou ENTRYPOINT dans ce Dockerfile ?**
> Parce que mon image hérite de ceux de l'image de base. `nginx:1.27-alpine` définit déjà `ENTRYPOINT ["/docker-entrypoint.sh"]` et `CMD ["nginx", "-g", "daemon off;"]` (vérifié avec `docker image inspect nginx:1.27-alpine`, et on retrouve les mêmes valeurs dans `monsite:1.0`). Je n'ai fait qu'ajouter des fichiers par-dessus, la façon de démarrer nginx ne change pas, donc pas besoin de les redéfinir.

> **Question : pourquoi la page est-elle toujours présente après la suppression et la recréation du conteneur, contrairement à la partie 1 ?**
> En partie 1, `docker cp` copiait la page dans la couche en écriture du conteneur. Cette couche est supprimée avec le conteneur (`docker rm`), et le nouveau conteneur repartait de l'image nginx d'origine. Ici la page est copiée dans l'image elle-même par l'instruction `COPY site/ /usr/share/nginx/html/` au moment du build. Elle fait partie des couches en lecture seule de `monsite:1.0`, donc chaque conteneur créé à partir de cette image a la page dès le départ.

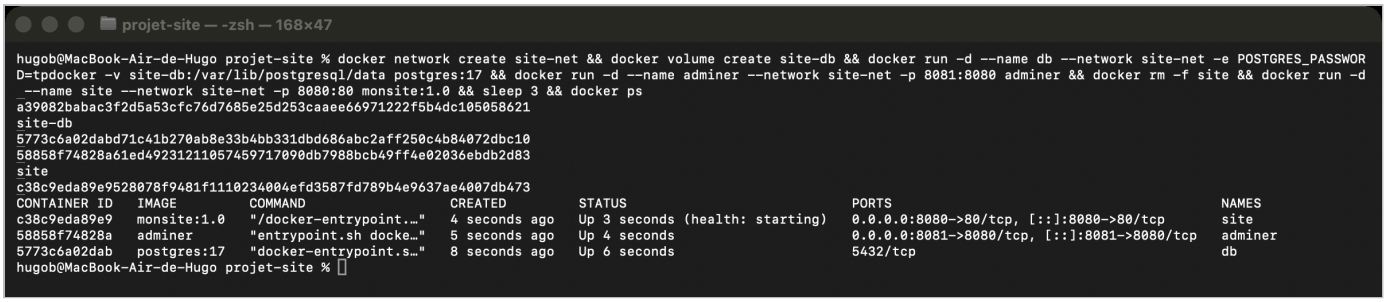
Bonus : conteneur site-dev avec un bind mount



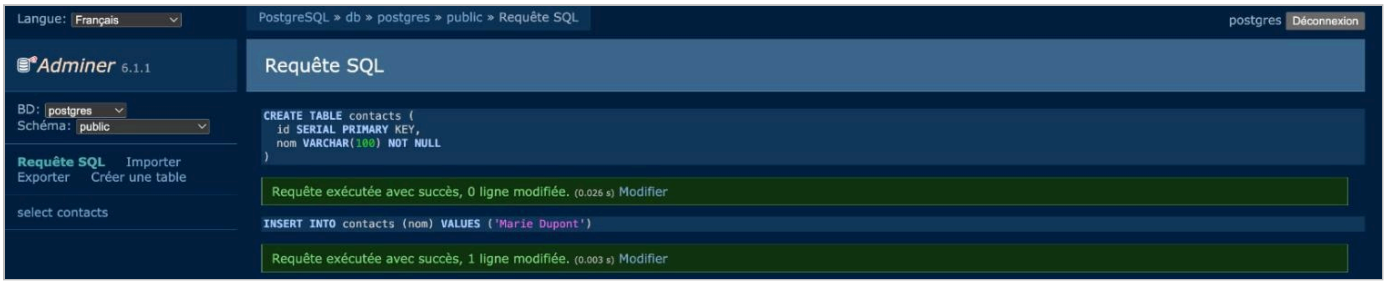
site-dev (port 8090) monte ./site en lecture seule (:ro). Après modification du h1 dans index.html sur l'hôte, site-dev affiche tout de suite la nouvelle version sans rebuild, alors que site (port 8080, page dans l'image) affiche toujours l'ancienne.

Le bind mount remplace le dossier /usr/share/nginx/html de l'image par mon dossier site/ de l'hôte : pratique pour développer. J'ai remis la page d'origine et supprimé site-dev après le test.

Étape 2 - La base de données



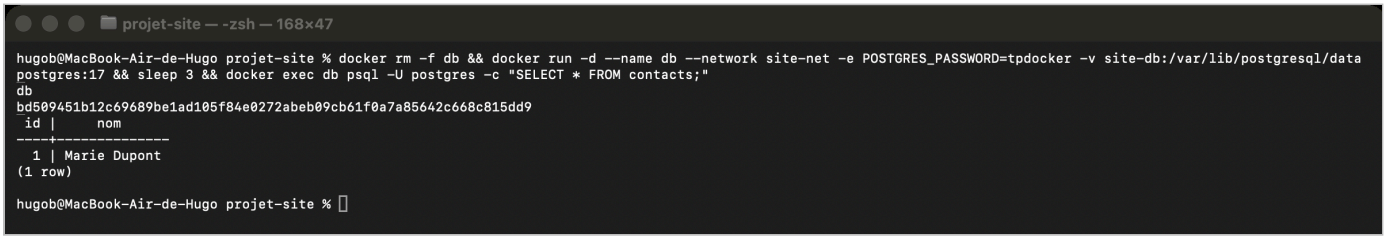
Création du réseau site-net et du volume site-db, lancement de db (postgres:17, port non publié), d'adminer (8081:8080) et recréation de site sur site-net. Dans docker ps, db affiche juste 5432/tcp (pas de 0.0.0.0:...).



Dans Adminer (http://localhost:8081), connecté au serveur db avec l'utilisateur postgres : création de la table contacts (id, nom) et insertion d'une ligne.



Adminer connecté au serveur db (fil d'Ariane PostgreSQL » db » postgres » public) affichant la table contacts et sa ligne.



Suppression du conteneur db puis recréation avec exactement la même commande : la ligne de contacts est toujours là (vérifié avec psql).



La table contacts et sa ligne sont toujours présentes dans Adminer après la recréation du conteneur db.



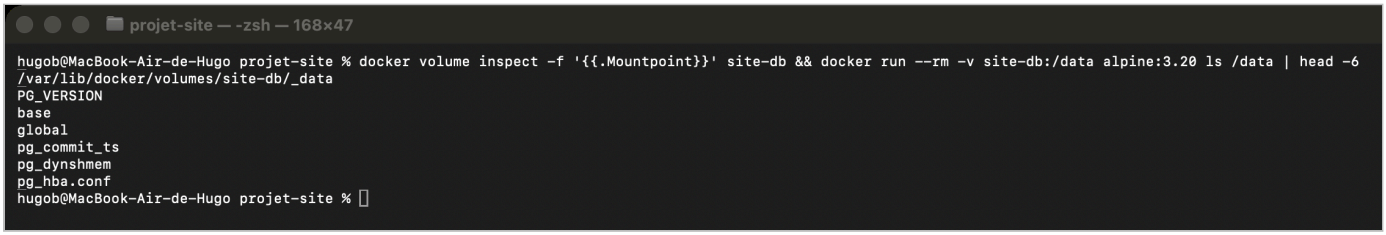
docker network inspect site-net liste les trois conteneurs (adminer, db, site) avec leur IP. docker port db ne renvoie rien : le port de la base n'est pas publié sur l'hôte.

J'ai filtré la sortie de `docker network inspect` avec grep car le JSON complet fait 66 lignes et ne tenait pas dans une capture.



ping db depuis un conteneur sur site-net (OK) puis depuis un conteneur sur le bridge par défaut (nom inconnu).

> **Question : pourquoi Adminer peut-il joindre la base avec le nom db ? Cela fonctionnerait-il sur le bridge par défaut ?**
> Les conteneurs sont sur un réseau bridge créé par moi (`site-net`). Sur ce type de réseau Docker fournit un DNS interne : chaque conteneur est enregistré sous son nom, donc `db` est résolu en 172.18.0.2. Adminer n'a pas besoin de connaître l'IP, qui peut d'ailleurs changer à chaque recréation du conteneur.
> Sur le bridge par défaut ça ne marche pas : il n'y a pas de résolution par nom, seulement par IP (ou avec l'ancienne option `--link`). Je l'ai vérifié avec un ping (capture ci-dessus) : sur site-net, `db` répond ; sur le bridge par défaut, « bad address 'db' ».



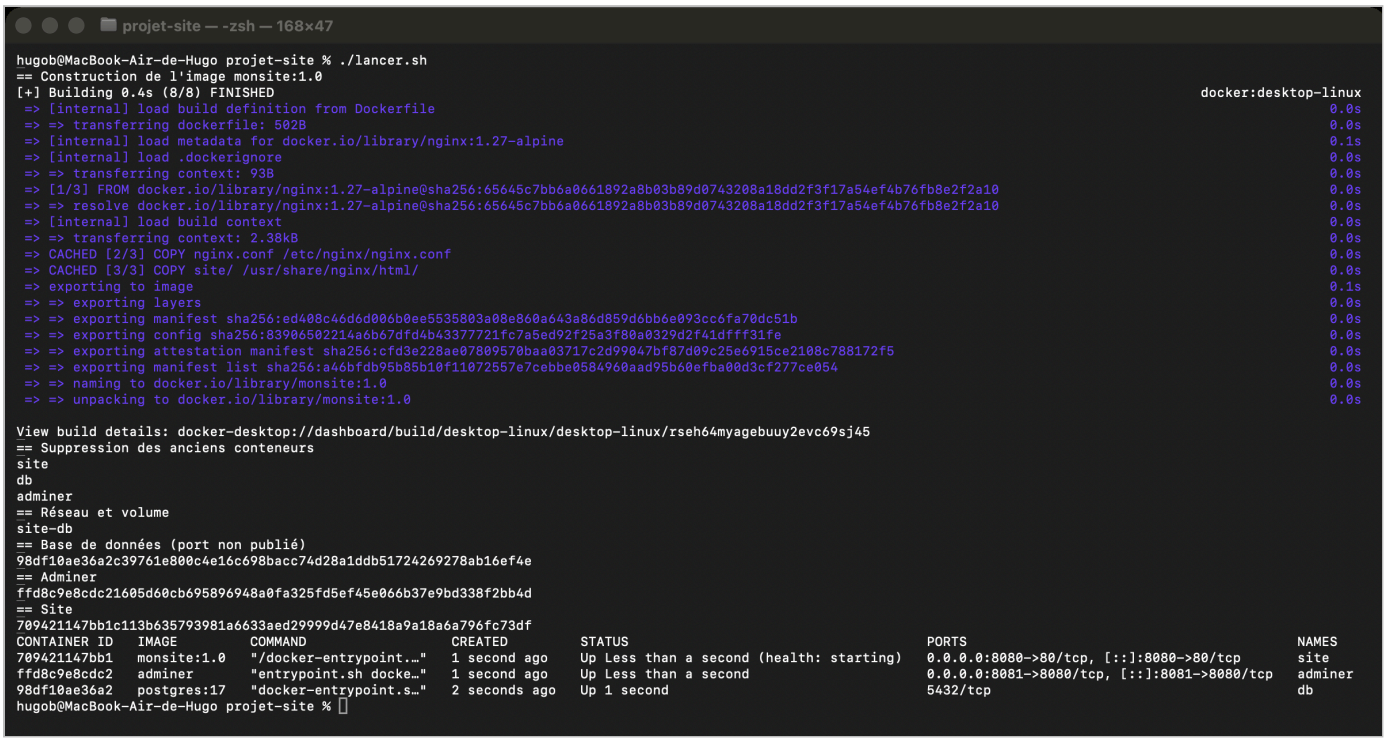
Point de montage du volume site-db et début de son contenu (fichiers de données PostgreSQL).

Question : où les données de la base sont-elles stockées sur l'hôte ? Comment l'avez-vous trouvé ?

Dans le volume nommé `site-db`. Avec `docker volume inspect site-db` on trouve son point de montage : `/var/lib/docker/volumes/site-db/_data`. Comme je suis sur Mac avec Docker Desktop, ce chemin est dans la machine virtuelle Linux de Docker et pas directement dans le Finder. Pour voir le contenu j'ai monté le volume dans un conteneur alpine : on y retrouve les fichiers de PostgreSQL (PG_VERSION, base, global...).

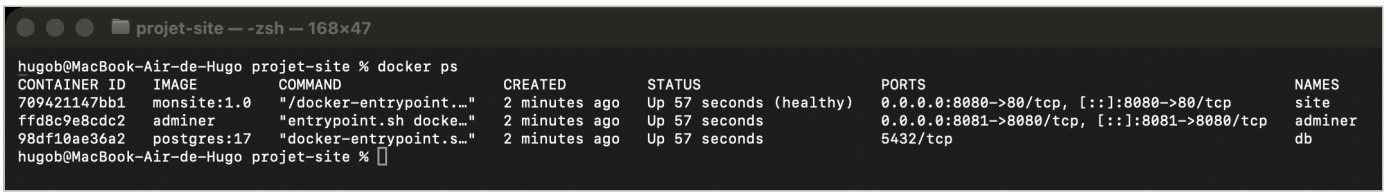
Étape 3 - La mise en production

J'ai écrit le script `lancer.sh` (contenu complet en annexe) qui reconstruit l'image, supprime les anciens conteneurs, crée le réseau et le volume s'ils n'existent pas, puis lance les trois conteneurs avec `--restart unless-stopped`, une rotation des logs (`--log-opt max-size=10m --log-opt max-file=3`), site limité à 128 Mo et 0,5 CPU, db limitée à 512 Mo.

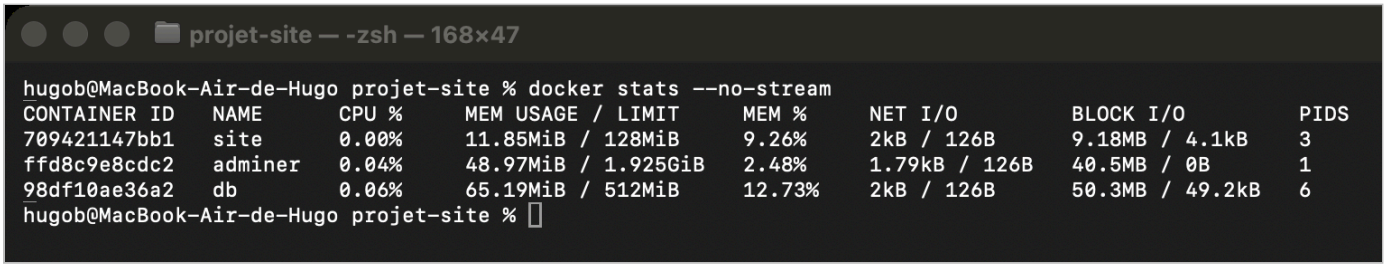


Exécution de ./lancer.sh : build de monsite:1.0, suppression des anciens conteneurs, réseau, volume et lancement des trois conteneurs.

J'ai ensuite redémarré Docker Desktop (`docker desktop restart`, l'équivalent sur Mac de `systemctl restart docker`).



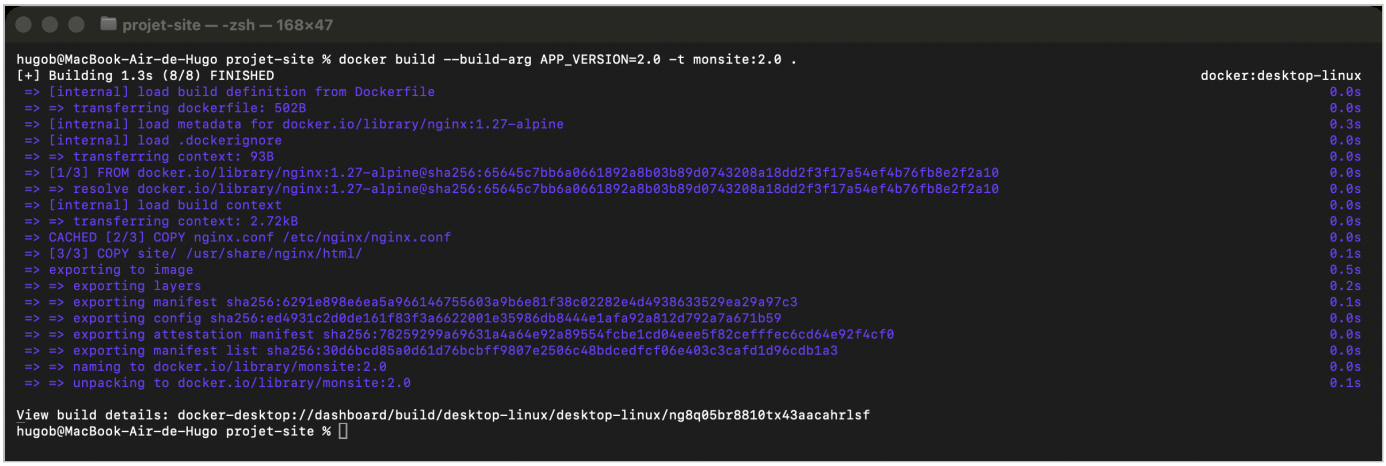
docker ps après le redémarrage de Docker Desktop : les trois conteneurs ont été créés il y a 2 minutes mais sont « Up 57 seconds » : ils ont été relancés tout seuls au redémarrage grâce à unless-stopped (site est de nouveau healthy).



docker stats --no-stream : limite mémoire de 128 MiB pour site et 512 MiB pour db (adminer n'a pas de limite, il voit toute la mémoire de la VM).

Déploiement de la version 2.0

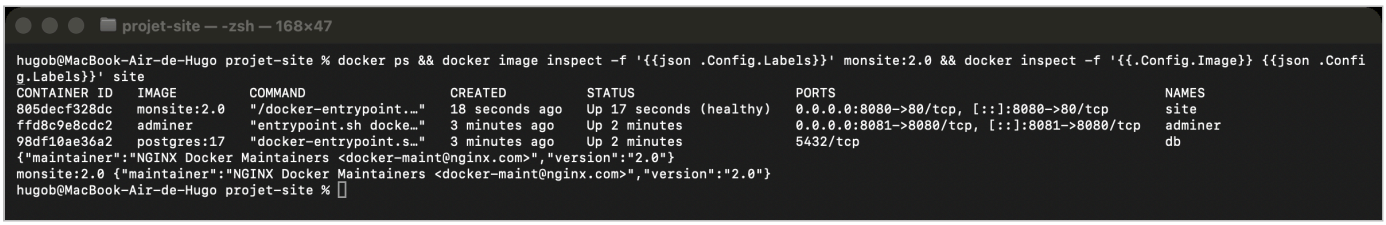
J'ai modifié `site/index.html` (encadré « Nouveau » et « version 2.0 » dans le pied de page), construit `monsite:2.0` puis remplacé uniquement le conteneur site, avec les mêmes options que dans le script. db et adminer n'ont pas été touchés.



Construction de monsite:2.0 avec APP_VERSION=2.0 (cette fois le COPY site/ n'est pas en cache car index.html a changé).



Remplacement du conteneur site seul par la version 2.0.



site tourne maintenant sur monsite:2.0 (healthy), adminer et db sont toujours les mêmes conteneurs (créés 3 minutes avant). Le label version vaut 2.0.



La nouvelle page (version 2.0) sur http://localhost:8080.



La table contacts est toujours visible dans Adminer après la montée de version.

Question : quelle est la différence entre les politiques always et unless-stopped ?
Les deux redémarrent le conteneur s'il s'arrête tout seul (plantage) et au redémarrage du démon Docker. La différence vient d'un arrêt manuel : avec `always`, un conteneur arrêté avec `docker stop` redémarre quand même au prochain redémarrage du démon ; avec `unless-stopped`, s'il a été arrêté à la main il reste arrêté. C'est plus logique en production : si j'arrête un service exprès, il ne revient pas tout seul.

Question : pourquoi limiter la mémoire d'un conteneur ? Que se passe-t-il s'il dépasse sa limite ?
Tous les conteneurs partagent la mémoire de la même machine. Sans limite, un seul conteneur (fuite mémoire, pic de trafic, grosse requête) peut prendre toute la RAM et faire tomber les autres services, voire l'hôte. La limite protège les autres conteneurs.
Si le conteneur dépasse sa limite, le noyau Linux (OOM killer) tue son processus : le conteneur s'arrête avec le code 137 et `docker inspect` indique `OOMKilled: true`. Avec `unless-stopped`, Docker le relance ensuite automatiquement.

Question : pourquoi les données de la base n'ont-elles pas été perdues lors de la montée de version ?

D'abord parce que je n'ai remplacé que le conteneur site : db n'a pas été touché. Et surtout, les données de PostgreSQL ne sont pas dans le conteneur mais dans le volume nommé `site-db`, monté sur `/var/lib/postgresql/data`. Un volume a une vie indépendante des conteneurs : même quand db est supprimé et recréé (étape 2, ou avec lancer.sh), il retrouve ses données en remontant le même volume.

Annexe - Fichiers du projet

Arborescence

```
projet-site/
├── .dockerignore
├── Dockerfile
├── commandes.md
├── lancer.sh
├── nginx.conf
└── site/
    └── index.html
```

Dockerfile

```
FROM nginx:1.27-alpine

# version de l'application, passée au build avec --build-arg
ARG APP_VERSION=dev
LABEL version=$APP_VERSION

# configuration nginx récupérée en partie 1
COPY nginx.conf /etc/nginx/nginx.conf
# contenu du site
COPY site/ /usr/share/nginx/html/

EXPOSE 80

# toutes les 10 secondes on vérifie que la page répond
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -q --spider http://localhost/ || exit 1
```

.dockerignore

```
commandes.md
.git
lancer.sh
Dockerfile
.dockerignore
```

lancer.sh

```
#!/bin/bash
# Recrée toute l'application à partir de zéro :
# image du site, réseau, volume et les trois conteneurs (site, db, adminer).
#
# Usage : ./lancer.sh [version]      (par défaut 1.0)

VERSION=${1:-1.0}
DB_PASSWORD=tpdocker

# options communes : redémarrage auto + rotation des logs (3 fichiers de 10 Mo max)
OPTIONS="--restart unless-stopped --log-opt max-size=10m --log-opt max-file=3"

echo "== Construction de l'image monsite:$VERSION"
docker build --build-arg APP_VERSION=$VERSION -t monsite:$VERSION . || exit 1

echo "== Suppression des anciens conteneurs"
docker rm -f site db adminer 2>/dev/null

echo "== Réseau et volume"
docker network inspect site-net > /dev/null 2>&1 || docker network create site-net
docker volume create site-db

echo "== Base de données (port non publié)"
docker run -d --name db --network site-net $OPTIONS \
  -m 512m \
  -e POSTGRES_PASSWORD=$DB_PASSWORD \
  -v site-db:/var/lib/postgresql/data \
  postgres:17

echo "== Adminer"
docker run -d --name adminer --network site-net $OPTIONS \
  -p 8081:8080 \
  adminer

echo "== Site"
docker run -d --name site --network site-net $OPTIONS \
  -m 128m --cpus 0.5 \
  -p 8080:80 \
  monsite:$VERSION

docker ps
```